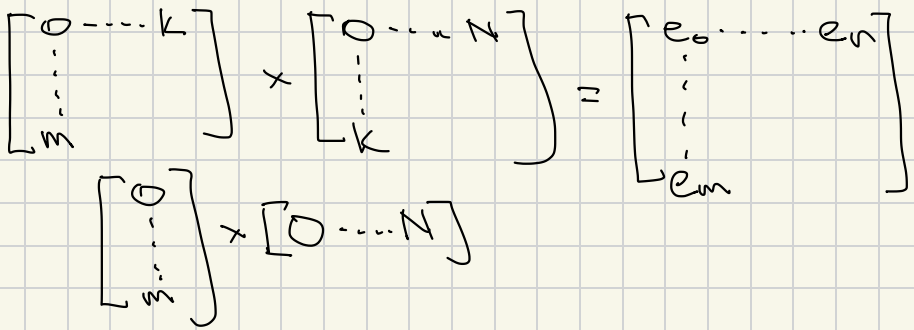
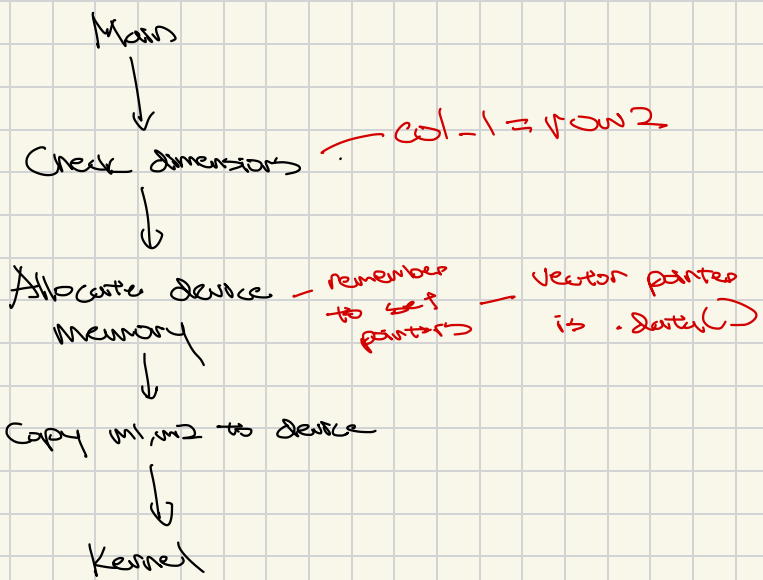
Matrix Mult Design - Basic

Inputs
- Matrix m1 [M x K] vector
- Matrix m2 [K x N] vector
- Row of m1
- Col of m2 rest can be derived

Output
- Matrix m3 [M x N] vector



M x N threads total for each element

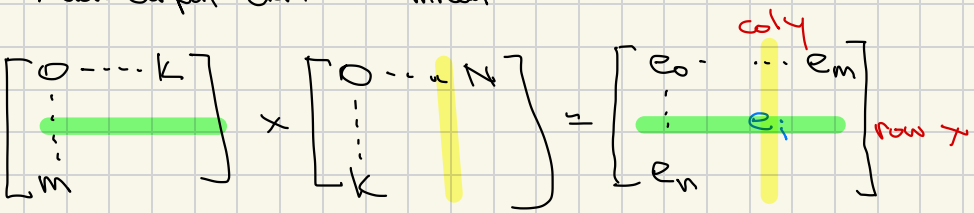


Matrix Mult kernel Simple

Inputs
- Pointers to m1, m2, m3
- Dimensions

Writes output element to m3

Each output elem => thread

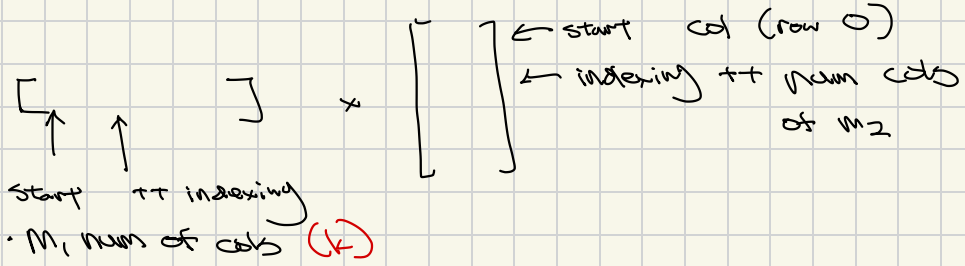


Given some e_i in row x & col y

$e_i = m1_{row\,x} \cdot m2_{col\,y}$

row = i / num cols

col = i % num cols



Should be 1 thread per result matrix element - max 1024 threads

Num blocks = elements + (num threads - 1) / num threads

↳ learn better ways to synchronize

num across blocks

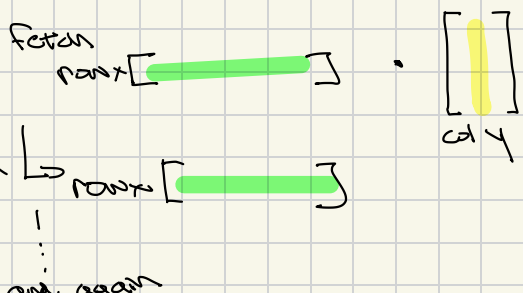
ceiling division

Nsight Observations from base - no optimization

- Testing using 512 x 512 matrix mult

Findings:
- 94% Achieved Occupancy
  - Good, GPU busy
- 80% Mem Throughput vs 80% Compute Throughput
  - Good, balanced, neither is bottlenecking other
- 1% Dram throughput, 81% L1 throughput, 68% L2 throughput
  - Good, data is mostly stored in cache vs ram
    - better for memory locality
- Branch Efficiency 100%, Avg Divergent Branches: 0
  - Good, no warp divergence

Issues
- Scheduler Statistics: 61% no eligible
  - Threads only issuing instructions only 39% of cycles
    - Waiting on memory to come back *
  - Problem
    - A lot of redundant fetching as each col/row is pulled multiple times



- Uncoalesced memory access 3,948% est speedup
  - Comes from b_vals[m1_row * k + i]
    - lot of mem addresses away

* Next Steps: 1. Shared memory and tiling

Some more benchmarking

Tested base kernel vs cuBLAS (Nvidia Standard)

Tried 32, 64, 128, 256, 512, 1024 num of threads launched across 512×512, 2048×2048, 4096×4096 matrices

Observations

- Kernel actually beats cuBLAS at 512×512
  - Prob just more lightweight kernel
  - cuBLAS initialization overhead
    - Had to warm up first

- Execution time around 50% of cuBLAS at 2048×2048
  25-30% of cuBLAS at 4096×4096

- Thread count doesn't really affect execution time
  - Best performance: 64 threads at 512×512
    256 threads at 2048×2048
    512 threads at 4096×4096

↑ This is prob due to being memory bound over compute bound

Scheduler can schedule more threads but they're all waiting on memory as seen from the weight

Solution: Reduce access to global memory

Roofline Analysis

- 4096 x 4096 at 512 threads

One element (i,j) = $\sum_{k=0}^{4096} A[i,k] \times B[k,j]$

so 4096 x 2 FLOPs per output elem

(2: addition, multiplication)

FLOPs = 4096x4096 (Total # of output elements) x 2x4096 (FLOPs per output element) = 137.44 GFLOPs

Bytes moved: 7.47 GB (Read) + 944.52 MB (Write) = 8.4 GB

Time taken: 81.02 ms — from ncu

Compute Throughput: $\frac{137.44\text{ GFLOPs}}{81.02\text{ ms}} = 1.696\text{ TFLOPs/s}$

Compute Time Floor: $\frac{137.44\text{ GFLOPs}}{56\text{ TFLOPs/s}} = 2.45\text{ ms}$ (max of 5080)

DRAM Memory Throughput: $\frac{8.4\text{ GB}}{81.02\text{ ms}} = 103.8\text{ GB/s}$

DRAM Memory Time Floor: $\frac{8.4\text{ GB}}{960\text{ GB/s}} = 8.75\text{ ms}$

L2 Time Floor: $\frac{276.25\text{ GB}}{3.4\text{ TB/s} / 0.94} = 76.3\text{ ms}$

(3.4 TB/s: Achieved BW; 0.94: percentage of peak BW)

$\frac{276.25\text{ GB}}{3.4\text{ TB/s}} = 81.0\text{ ms}$ — very close to t actual

confirmed L2 is bottleneck